

TensorPoW: A Tensor-Native Proof-of-Work Currency

Author: Arav Jain (@aravhawk)

Draft date: 2026-05-27

Abstract

TensorPoW is a permissionless proof-of-work currency whose work function is deterministic signed INT8 matrix multiplication with INT32 accumulation. The chain separates high-rate transaction carriers, called fruits, from lower-rate ordering checkpoints, called anchors. Fruits carry transactions under a permanent work target. Anchors commit fruit sets, parent candidates, shard-tree state, and fee-floor state under a WTEMA-adjusted target. The ledger is a pure UTXO system: balances are derived only from unspent outputs, and genesis creates no spendable allocation.

The design goal is to make useful parallel tensor hardware the natural mining substrate while keeping consensus byte-exact across CUDA, MPS, x86 CPU, and ARM CPU implementations. All consensus hashes use BLAKE3 with explicit domain separation, all consensus integers are little-endian, and every consensus object has a canonical byte encoding.

1. Introduction

Bitcoin demonstrated that an open network can order payments by making history expensive to rewrite. TensorPoW keeps that central idea but changes the work primitive and the block structure. Modern compute supply is increasingly shaped by tensor accelerators. TensorPoW therefore uses a fixed 1024 by 1024 signed INT8 matrix multiplication as its proof-of-work kernel. The output is serialized as signed little-endian INT32 cells and hashed with BLAKE3.

TensorPoW is also a BlockDAG rather than a single-parent chain. Fruits can reference multiple recent parents so honest work produced during propagation races remains useful. Anchors periodically commit ordered fruit sets and the state needed by wallets, miners, and light clients. The currency unit is TSC; the atomic subunit is the matom, with 100,000,000 matoms per TSC.

This paper describes the permanent protocol semantics. It does not introduce a numbered-generation network. Future incompatible rule changes require planned hardforks of the same chain.

2. System Model

Nodes communicate over libp2p using TCP and QUIC transports, Noise security, yamux multiplexing, Kademlia peer discovery, and Gossipsub 2.0 topic gossip. The protocol defines gossip topics for fruits, anchors, and shard transactions. Every wire message begins with the TensorPoW wire magic, a typed payload length, the canonical payload, and a short BLAKE3-derived checksum. A malformed network message is dropped before its payload can affect consensus.

The ledger is a pure UTXO ledger. There are no accounts, account nonces, smart contract storage slots, or shared mutable balances. A transaction consumes existing outpoints and creates new outputs. A transaction is valid only when the referenced outputs exist, are unspent, satisfy their script templates, and obey the active shard fee floor.

The network assumes asynchronous propagation and adversarial peers. It does not enforce a consensus bandwidth minimum. Public routing is expected to work best for operators that provision at least the recommended 10 MB/s bandwidth.

3. Cryptographic Commitments

Every consensus hash is BLAKE3 with a 32-byte output unless the protocol explicitly calls for BLAKE3 XOF. Hash preimages begin with a one-byte domain constant followed by canonical bytes. Domain separation is used for PoW challenges, PoW outputs, fruit headers, anchor headers, transaction IDs, signature hashes, outpoints, UTXO values, Merkle nodes, addresses, genesis, shard trees, fee floors, and data-availability samples.

Ed25519 is the only active signature type at genesis. Transaction inputs carry a signature type, and verifiers dispatch through a fixed signature table. Unknown signature types fail validation. The reserved post-quantum signature identifier does not validate until a hardfork activates rules for it.

Tensorcoin addresses are lowercase Bech32m strings with the human-readable part 'tsc'. The payload is 'BLAKE3(DOMAIN_ADDRESS || ed25519_public_key)', converted from 8-bit bytes to 5-bit Bech32m words. Mixed case, wrong HRP, invalid checksums, invalid padding, and wrong decoded payload lengths are invalid.

4. Tensor Proof of Work

The PoW kernel is:

'''text

C[row, col] = sum(A[row, k] * B[k, col]) for k in [0, 1024)

'''

'A' and 'B' are signed INT8 matrices. 'C' is a signed INT32 matrix. The accumulator must not wrap, and floating-point promotion is not a valid consensus implementation. Canonical output bytes serialize every INT32 cell in